Metodologie di Programmazione (A-L)

II semestre - a.a. 2023 – 2024

Parte 12 - Input/Output**

a cura di Mattia Samory*

DIPARTIMENTO
DI INFORMATICA

SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

📣Avvisi

- Esercitazioni 22/5 e 29/5 con i tutor
- Logistica esame

Letture

- I/O (Horstmann 11.1–4)

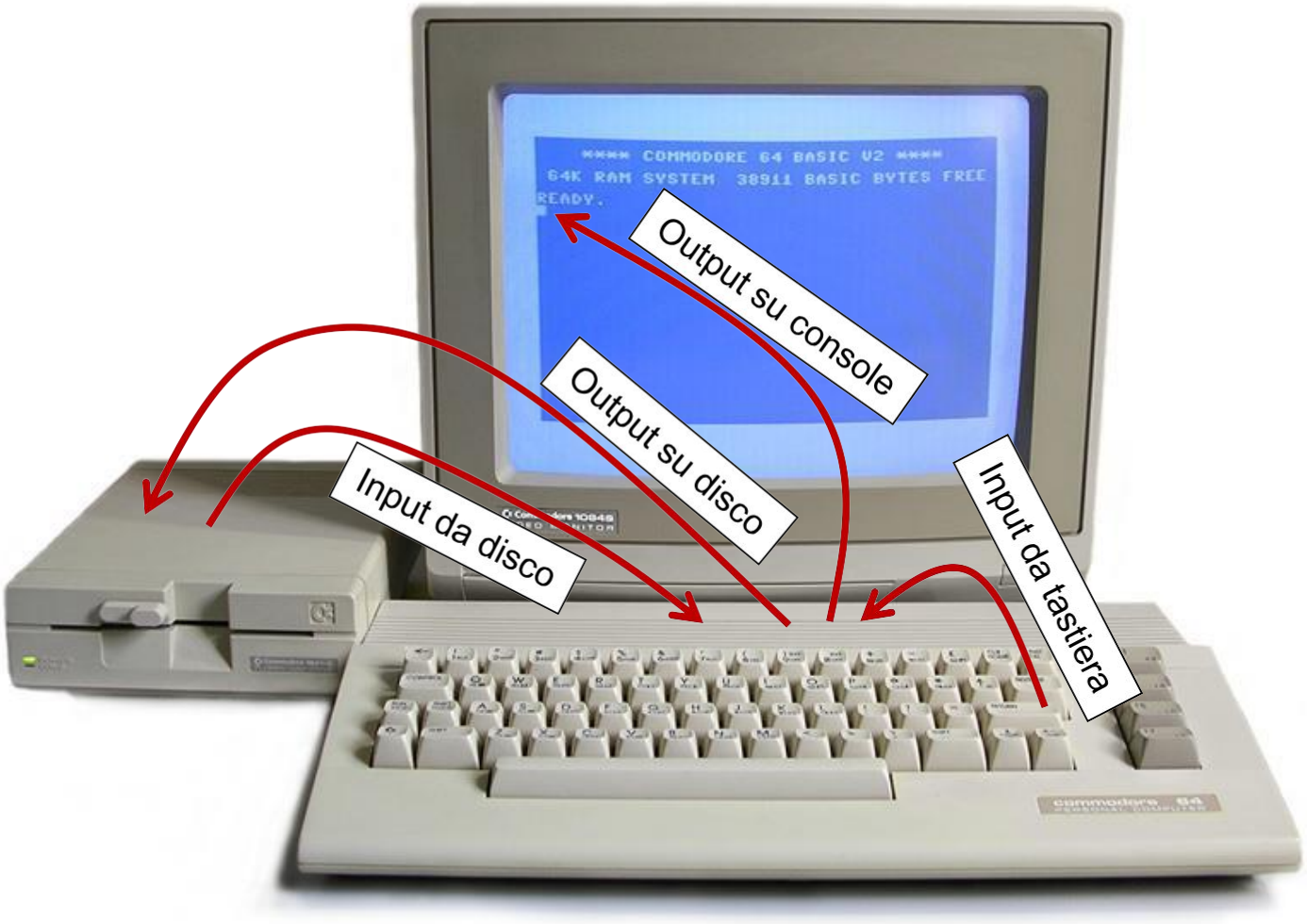
🔄 Recap

-

🎯Obiettivi

- Terminale: Print/Output/InputStream, Scanner
- File: testo, binari
- Stream
- Gestione delle risorse
- Java NIO
- Serializzazione

Input & Output





Output su console

- Usando la classe System.out (oggetto dello standard output)
- E' un campo statico, pubblico, final di System di tipo java.io.PrintStream (estende java.io.OutputStream)

```
String s = "hello";
System.out.print(s);
System.out.println();
System.out.println(10000000000L);
System.out.print(42.0);
System.out.format("La stringa contiene %d caratteri; il primo carattere è %c\n", s.length(), s.charAt(0));
```

java.io.PrintStream

protected void	**clearError**() Clears the internal error state of this stream.
void	**close**() Closes the stream.
void	**flush**() Flushes the stream.
PrintStream	**format**(Locale l, String format, Object... args) Writes a formatted string to this output stream using the specified format string and arguments.
PrintStream	**format**(String format, Object... args) Writes a formatted string to this output stream using the specified format string and arguments.
void	**print**(boolean b) Prints a boolean value.
void	**print**(char c) Prints a character.
void	**print**(char[] s) Prints an array of characters.
void	**print**(double d) Prints a double-precision floating-point number.
void	**print**(float f) Prints a floating-point number.
void	**print**(int i) Prints an integer.
void	**print**(long l) Prints a long integer.
void	**print**(Object obj) Prints an object.
void	**print**(String s) Prints a string.



Input da tastiera

- Usando la classe java.util.Scanner costruita con la standard input stream:
- System.in è un campo statico, pubblico, final di System di tipo java.io.InputStream

```
Scanner s = new Scanner(System.in);
int k = s.nextInt();
System.out.println("Hai digitato: "+k);
String string = s.next();
System.out.println("Hai digitato: "+string);
```

java.io.InputStream

Class InputStream

java.lang.Object
    java.io.InputStream

All Implemented Interfaces:

`Closeable, AutoCloseable`

Direct Known Subclasses:

`AudioInputStream, ByteArrayInputStream, FileInputStream, FilterInputStream, InputStream, ObjectInputStream, PipedInputStream, SequenceInputStream, StringBufferInputStream`

```
public abstract class InputStream
extends Object
implements Closeable
```

This abstract class is the superclass of all classes representing an input stream of bytes.

Applications that need to define a subclass of `InputStream` must always provide a method that returns the next byte of input.

Since:

`JDK1.0`

See Also:

`BufferedInputStream, ByteArrayInputStream, DataInputStream, FilterInputStream, read(), OutputStream, PushbackInputStream`

Constructor Summary

Constructors

Constructor and Description
`InputStream()`

Method Summary

All Methods | **Instance Methods** | **Abstract Methods** | **Concrete Methods**

Modifier and Type	Method and Description
`int`	`available()` Returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
`void`	`close()` Closes this input stream and releases any system resources associated with the stream.
`void`	`mark(int readlimit)` Marks the current position in this input stream.
`boolean`	`markSupported()` Tests if this input stream supports the mark and reset methods.
`abstract int`	`read()` Reads the next byte of data from the input stream.
`int`	`read(byte[] b)`

La classe java.util.Scanner (1)

- Usando la classe java.util.Scanner costruita con la standard input stream:

Constructor Summary
`Scanner(File source)` Constructs a new `Scanner` that produces values scanned from the specified file.
`Scanner(File source, String charsetName)` Constructs a new `Scanner` that produces values scanned from the specified file.
`Scanner(InputStream source)` Constructs a new `Scanner` that produces values scanned from the specified input stream.
`Scanner(InputStream source, String charsetName)` Constructs a new `Scanner` that produces values scanned from the specified input stream.
`Scanner(Readable source)` Constructs a new `Scanner` that produces values scanned from the specified source.
`Scanner(ReadableByteChannel source)` Constructs a new `Scanner` that produces values scanned from the specified channel.
`Scanner(ReadableByteChannel source, String charsetName)` Constructs a new `Scanner` that produces values scanned from the specified channel.
`Scanner(String source)` Constructs a new `Scanner` that produces values scanned from the specified string.

La classe java.util.Scanner (2)

- Alcuni metodi della classe:

Method Summary	
void	close() Closes this scanner.
Pattern	delimiter() Returns the Pattern this Scanner is currently using to match delimiters.
String	findInLine(Pattern pattern) Attempts to find the next occurrence of the specified pattern ignoring delimiters.
String	findInLine(String pattern) Attempts to find the next occurrence of a pattern constructed from the specified string, ignoring delimiters.
String	findWithinHorizon(Pattern pattern, int horizon) Attempts to find the next occurrence of the specified pattern.
String	findWithinHorizon(String pattern, int horizon) Attempts to find the next occurrence of a pattern constructed from the specified string, ignoring delimiters.
boolean	hasNext() Returns true if this scanner has another token in its input.
boolean	hasNext(Pattern pattern) Returns true if the next complete token matches the specified pattern.
boolean	hasNext(String pattern) Returns true if the next token matches the pattern constructed from the specified string.

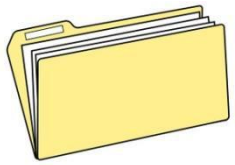
La classe java.util.Scanner (3)

- Alcuni metodi della classe:

double	nextDouble() Scans the next token of the input as a double.
float	nextFloat() Scans the next token of the input as a float.
int	nextInt() Scans the next token of the input as an int.
int	nextInt(int radix) Scans the next token of the input as an int.
String	nextLine() Advances this scanner past the current line and returns the input that was skipped.
long	nextLong() Scans the next token of the input as a long.
long	nextLong(int radix) Scans the next token of the input as a long.
short	nextShort() Scans the next token of the input as a short.
short	nextShort(int radix) Scans the next token of the input as a short.
int	radix() Returns this scanner's default radix.
void	remove() The remove operation is not supported by this implementation of Iterator.
Scanner	reset() Resets this scanner.

🎯Obiettivi

- ✅ Terminale: Print/Output/InputStream, Scanner
  - File: testo, binari
  - Stream
  - Gestione delle risorse
  - Java NIO
  - Serializzazione



I file

- Un file è una collezione di dati salvata su supporto di memorizzazione di massa
- Un file di dati non è parte del codice sorgente di un programma!
- Lo stesso file può essere letto o modificato da programmi differenti
  - Le funzioni di base sono fornite dal sistema operativo
- Il programma deve conoscere il formato dei dati nel file
- E' IMPORTANTE distinguere tra file di testo e file binari



I file di testo

- Un file di testo contiene linee di testo (ad esempio, in ASCII)
- Ogni linea termina con un carattere di nuova linea (“\n”) o carriage return (“\r”) concatenato con nuova linea
- Esempi:
  - Testo puro (mio_file.txt)
  - Sorgenti di programmi (MiaClasse.java)
  - Documenti HTML (index.html)
  - File di dati testuali gestiti da alcuni programmi (es. std.ics di korganizer)



I file binari

- Un file binario può contenere qualsiasi informazione sotto forma di concatenazione di byte
- Solo il programmatore/progettista sa come interpretarlo
- Diversi programmi potrebbero interpretare lo stesso file in modo diverso (ad esempio, un'immagine)
- Esempi:
  - Programmi compilati (es. MiaClasse.class)
  - File di immagini (es. schermata.gif)
  - File musicali (es. musica.mp3)
- Qualsiasi file può essere trattato come file binario
  - Anche i file di testo!

La classe java.io.File

Contiene il separatore tra percorsi come variabili d'ambiente (tipicamente ; o :)

Contiene il separatore di percorsi (/ o \, secondo il sistema operativo)

Field Summary

static String	**pathSeparator** The system-dependent path-separator character, represented as a string for convenience.
static char	**pathSeparatorChar** The system-dependent path-se
static String	**separator** The system-dependent default name-separator character, represented as a string for convenience.
static char	**separatorChar** The system-dependent default name-separator character.

Constructor Summary

File(File parent, String child) Creates a new File instance from a parent abstract pathname and a child pathname string.
File(String pathname) Creates a new File instance by converting the given pathname string into an abstract pathname.
File(String parent, String child) Creates a new File instance from a parent pathname string and a child pathname string.
File(URI uri) Creates a new File instance by converting the given file: URI into an abstract pathname.

Method Summary

boolean	**canExecute**() Tests whether the application can execute the file denoted by this abstract pathname.
boolean	**canRead**() Tests whether the application can read the file denoted by this abstract pathname.
boolean	**canWrite**() Tests whether the application can modify the file denoted by this abstract pathname.
int	**compareTo**(File pathname) Compares two abstract pathnames lexicographically.
boolean	**createNewFile**() Atomically creates a new, empty file named by this abstract pathname if and only if a file with this name does not yet exist.
static File	**createTempFile**(String prefix, String suffix) Creates an empty file in the default temporary-file directory, using the given prefix and suffix to generate its name.

La classe java.io.File

boolean	**exists**() Tests whether the file or directory denoted by this abstract pathname exists.
File	**getAbsoluteFile**() Returns the absolute form of this abstract pathname.
String	**getAbsolutePath**() Returns the absolute pathname string of this abstract pathname.
File	**getCanonicalFile**() Returns the canonical form of this abstract pathname.
String	**getCanonicalPath**() Returns the canonical pathname string of this abstract pathname.
long	**getFreeSpace**() Returns the number of unallocated bytes in the partition named by this abstract path name.
String	**getName**() Returns the name of the file or directory denoted by this abstract pathname.
String	**getParent**() Returns the pathname string of this abstract pathname's parent, or `null` if this pathname does not name a parent directory.
File	**getParentFile**() Returns the abstract pathname of this abstract pathname's parent, or `null` if this pathname does not name a parent directory.
String	**getPath**() Converts this abstract pathname into a pathname string.
long	**getTotalSpace**() Returns the size of the partition named by this abstract pathname.
long	**getUsableSpace**() Returns the number of bytes available to this virtual machine on the partition named by this abstract pathname.
int	**hashCode**() Computes a hash code for this abstract pathname.
boolean	**isAbsolute**() Tests whether this abstract pathname is absolute.
boolean	**isDirectory**() Tests whether the file denoted by this abstract pathname is a directory.
boolean	**isFile**() Tests whether the file denoted by this abstract pathname is a normal file.

Senza . né .. rispetto ad absolutePath

Accesso Unbuffered

lettura e scrittura dei dati direttamente dal disco al programma senza un intermediario.

- Classi Java: FileInputStream, FileOutputStream, FileReader, FileWriter
- Ogni lettura/scrittura è un'operazione I/O fisica.
- Inefficiente se ripetuta: costi elevati di I/O.

Accesso Buffered

buffer intermedio in memoria per accumulare dati prima di accesso al disco.

- Classi Java: BufferedInputStream, BufferedOutputStream, BufferedReader, BufferedWriter
- Diminuisce il numero di operazioni I/O fisiche aggregando i dati.
- Migliora le prestazioni riducendo il tempo di accesso al disco.
- Maggiore utilizzo di memoria dovuto al buffer intermedio.

👐TPS

Un servizio in tempo reale, come un server di applicazioni, deve scrivere log di diagnostica su un file. Il volume dei log può variare considerevolmente:

- Durante periodi normali, il servizio genera poche righe di log.
- Durante picchi di attività o situazioni di errore, il servizio può generare un elevato volume di log in breve tempo.

Usereste I/O buffered o unbuffered?

- Buffered
  - + meno I/O fisiche, migliori prestazioni durante i picchi di attività.
  - - ritardo nella scrittura effettiva dei log su disco: dati persi nel buffer se il servizio si arresta improvvisamente.
- Unbuffered
  - + riduce il rischio di perdita di dati in caso di arresto imprevisto.
  - - sovraccarico di molte operazioni I/O fisiche, rallentando potenzialmente il servizio.

Possibili Soluzioni

- passare da un buffered I/O durante i picchi di attività a un unbuffered I/O durante i periodi normali.
- ibrido: buffered con flush periodico
- …

🎯Obiettivi

- ✅ Terminale: Print/Output/InputStream, Scanner
- ✅ File: testo, binari
  - Stream
  - Gestione delle risorse
  - Java NIO
  - Serializzazione

Sugli stream

- Uno stream è un'astrazione derivata da dispositivi di input o output sequenziale
- Uno stream di input riceve uno stream di caratteri "uno alla volta"
- Uno stream di output produce uno stream di caratteri
- Gli stream non si applicano solo ai file, ma anche a dispositivi di input/output, internet, ecc.
- Un file può essere trattato come uno stream di input o output
  - In realtà i file sono bufferizzati per questioni di efficienza

Quattro gerarchie di classi in Java

- Per leggere o scrivere caratteri (16 bit per volta):
  - java.io.Reader/Writer
- Per leggere o scrivere byte (da file binari, 8 bit per volta)
  - java.io.InputStream/OutputStream
- L'accesso ai file di testo è stato semplificato in Java 5 mediante l'aggiunta di una classe java.util.Scanner
  - Che tuttavia è **più lenta** proprio perché **più potente**

Leggere caratteri: l'interfaccia java.io.Reader

All Methods	Instance Methods	Abstract Methods	Concrete Methods

Modifier and Type	Method and Description
abstract void	**close**() Closes the stream and releases any system resources associated with it.
void	**mark**(int readAheadLimit) Marks the present position in the stream.
boolean	**markSupported**() Tells whether this stream supports the mark() operation.
int	**read**() Reads a single character.
int	**read**(char[] cbuf) Reads characters into an array.
abstract int	**read**(char[] cbuf, int off, int len) Reads characters into a portion of an array.
int	**read**(**CharBuffer** target) Attempts to read characters into the specified character buffer.
boolean	**ready**() Tells whether this stream is ready to be read.
void	**reset**() Resets the stream.
long	**skip**(long n)

🎯Obiettivi

- ✅ Terminale: Print/Output/InputStream, Scanner
- ✅ File: testo, binari
- ✅ Stream
  - Gestione delle risorse
  - Java NIO
  - Serializzazione

FileReader e BufferedReader

- BufferedReader permette una lettura bufferizzata dei caratteri forniti da FileReader:

```
BufferedReader br = null;

try
{
    br = new BufferedReader(new FileReader(fileName));
    while(br.ready())
    {
        String line = br.readLine();
        // ...
    }
}
catch(IOException e)
{
    // gestisci l'eccezione
}
finally
{
    if (br != null)
        try { br.close(); } catch(IOException e) { /* gestisci */ }
    }
}
```

Try with resources

- E' possibile specificare tra parentesi dopo try un elenco di istruzioni (separate da ;) che definiscono **risorse da chiudere automaticamente**:

```
try(BufferedReader br = new BufferedReader(new FileReader(fileName)))
{
    while(br.ready())
    {
        String line = br.readLine();
        // ...
    }
}
catch(IOException e)
{
    // gestisci l'eccezione
}
```

Come è possibile chiudere automaticamente uno stream?

- Tutti gli oggetti di classi che implementano java.lang.AutoCloseable che è estesa dall'interfaccia java.io.Closeable

```
public interface AutoCloseable
{
    void close() throws Exception;
}
```

Scrivere caratteri: l'interfaccia java.io.Writer

All Methods	Instance Methods	Abstract Methods	Concrete Methods

Modifier and Type	Method and Description
`Writer`	**`append`**`(char c)` Appends the specified character to this writer.
`Writer`	**`append`**`(`**`CharSequence`**` csq)` Appends the specified character sequence to this writer.
`Writer`	**`append`**`(`**`CharSequence`**` csq, int start, int end)` Appends a subsequence of the specified character sequence to this writer.
`abstract void`	**`close`**`()` Closes the stream, flushing it first.
`abstract void`	**`flush`**`()` Flushes the stream.
`void`	**`write`**`(char[] cbuf)` Writes an array of characters.
`abstract void`	**`write`**`(char[] cbuf, int off, int len)` Writes a portion of an array of characters.
`void`	**`write`**`(int c)` Writes a single character.
`void`	**`write`**`(`**`String`**` str)` Writes a string.
`void`	**`write`**`(`**`String`**` str, int off, int len)`

FileWriter e BufferedWriter

- Usiamo il try with resources:

```
try(BufferedWriter bw = new BufferedWriter(new FileWriter(fileName)))
{
    bw.write("bla bla bla");
    // ...
}
catch(IOException e)
{
    // gestisci l'eccezione
}
```



Leggere un file di testo

- Un'alternativa (più lenta) è usare la classe java.util.Scanner:

```
File f = new File("mio_file.txt");

try
{
    Scanner in = new Scanner(f);

    // finche' esiste una prossima riga
    while(in.hasNext())
        // stampa la riga
        System.out.println(in.nextLine());

    // chiude lo scanner e il file
    in.close();
}
catch(FileNotFoundException e)
{
    e.printStackTrace();
}
```

Costruisce l'oggetto File

Costruisce lo scanner con il file

Itera sulle righe di testo del file

Prossima riga di testo

Chiude scanner e file



Scrivere un file di testo

- Si può utilizzare la classe PrintWriter e i metodi corrispondenti:

```
File f = new File("mio_file.txt");

try
{
    PrintWriter out = new PrintWriter(f);

    out.println("Prima riga del file");
    out.print("Seconda riga: ");
    out.print(2);

    // chiude il file
    out.close();
}
catch(FileNotFoundException e)
{
    e.printStackTrace();
}
```

Costruisce l'oggetto File

Costruisce il Writer di file

Scrive due righe di testo

Chiude il file



Esercizio: numerare un file

- Si progetti una classe i cui oggetti sono costruiti a partire da un determinato file di testo f
- La classe espone un metodo numera che:
  1. crea un file il cui nome è dato da quello del file f cui viene concatenata l'estensione ".num"
  2. contiene riga per riga le informazioni contenute nel file f cui viene anteposto il numero di riga
- Ad esempio, date le righe:

```
Questa è la prima riga del file
Questa è la seconda riga del file
```

- salva nel nuovo file:

```
1: Questa è la prima riga del file
2: Questa è la seconda riga del file
```

Scrivere un file di testo formattato

- E' possibile utilizzare la classe java.util.Formatter
  - "An interpreter for printf-style format strings"

```
public class FormattaFile
{
    private String nome;
    private int valore;

    public void scrivi(String filename) throws IOException
    {
        Formatter output = new Formatter(filename);
        output.format("%s\t%d", nome, valore);
        output.close();
    }
}
```

Leggere un file di testo formattato

- Essendo il testo nel file formattato usando dei separatori, lo Scanner è in grado di restituire la prossima stringa e il prossimo intero

```
public class FormattaFile
{
    private String nome;
    private int valore;

    public void scrivi(String filename) throws IOException
    {
        Formatter output = new Formatter(filename);
        output.format("%s\t%d", nome, valore);
        output.close();
    }

    public void leggi(String filename) throws IOException
    {
        Scanner input = new Scanner(new File(filename));
        nome = input.next();
        valore = input.nextInt();
    }
}
```

🎯Obiettivi

- ✅ Terminale: Print/Output/InputStream, Scanner
- ✅ File: testo, binari
- ✅ Stream
- ✅ Gestione delle risorse
  - Java NIO
  - Serializzazione

Per il nuovo codice, non utilizzate java.io.File

- Molti metodi di File non emettono eccezione quando falliscono
- Nessun supporto per i collegamenti simbolici
- Mancanza di metadati: **permessi**, **proprietario**, ecc.
- Molti metodi non scalavano su cartelle piene di file

- La mappatura completa delle funzionalità è disponibile qui:

http://docs.oracle.com/javase/tutorial/essential/io/legacy.html



java.nio.file.Path

- La classe java.io.File è rimpiazzata dall'interfaccia java.nio.file.Path che rappresenta un percorso gerarchico

Interface Path

All Superinterfaces:

Comparable<Path>, Iterable<Path>, Watchable

```
public interface Path
extends Comparable<Path>, Iterable<Path>, Watchable
```

An object that may be used to locate a file in a file system. It will typically represent a system dependent file path.

A `Path` represents a path that is hierarchical and composed of a sequence of directory and file name elements separated by a special separator or delimiter. A *root component*, that identifies a file system hierarchy, may also b
present. The name element that is *farthest* from the root of the directory hierarchy is the name of a file or directory. The other name elements are directory names. A `Path` can represent a root, a root and a sequence of names
or simply one or more name elements. A `Path` is considered to be an *empty path* if it consists solely of one name element that is empty. Accessing a file using an *empty path* is equivalent to accessing the default directory of th
file system. `Path` defines the `getFileName`, `getParent`, `getRoot`, and `subpath` methods to access the path components or a subsequence of its name elements.

In addition to accessing the components of a path, a `Path` also defines the `resolve` and `resolveSibling` methods to combine paths. The `relativize` method that can be used to construct a relative path between two path
Paths can be `compared`, and tested against each other using the `startsWith` and `endWith` methods.

This interface extends `Watchable` interface so that a directory located by a path can be `registered` with a `WatchService` and entries in the directory watched.

WARNING: This interface is only intended to be implemented by those developing custom file system implementations. Methods may be added to this interface in future releases.

Accessing Files

Paths may be used with the `Files` class to operate on files, directories, and other types of files. For example, suppose we want a `BufferedReader` to read text from a file "`access.log`". The file is located in a directory "`log`
relative to the current working directory and is UTF-8 encoded.

```
Path path = FileSystems.getDefault().getPath("logs", "access.log");
BufferReader reader = Files.newBufferedReader(path, StandardCharsets.UTF_8);
```

Ottenere un Path

- Per ottenere un Path è possibile utilizzare il metodo Paths.get

•Path p = Paths.get("/tmp/foo");

che è un'abbreviazione per:

- Path p = FileSystems.getDefault().getPath("/tmp/foot");

Importante: sui percorsi

- E' importante evitare l'utilizzo esplicito dei separatori di nomi / o \, quindi:
- O si costruisce il percorso passando le varie cartelle a legare:

  Paths.get(cartella1, cartella2, …, cartellaN, file)

- O si costruisce il percorso con File.separator:

  Paths.get(cartella1+File.separator+…+File.separator
  +cartellaN+File.separator+file);

- **Comunque:** i percorsi con / funzionano sia su linux che su Windows

java.nio.file.Files

- Le operazioni che prima si svolgevano nella classe File ora sono metodi statici della classe java.nio.file.Files

Class Files

java.lang.Object
    java.nio.file.Files

```
public final class Files
extends Object
```

This class consists exclusively of static methods that operate on files, directories, or other types of files.

In most cases, the methods defined here will delegate to the associated file system provider to perform the file operations.

Since:

1.7

Method Summary

Methods

Modifier and Type	Method and Description
static long	**copy**(**InputStream** in, **Path** target, **CopyOption**... options) Copies all bytes from an input stream to a file.
static long	**copy**(**Path** source, **OutputStream** out) Copies all bytes from a file to an output stream.
static **Path**	**copy**(**Path** source, **Path** target, **CopyOption**... options) Copy a file to a target file.
static **Path**	**createDirectories**(**Path** dir, **FileAttribute**<?>... attrs) Creates a directory by creating all nonexistent parent directories first.
static **Path**	**createDirectory**(**Path** dir, **FileAttribute**<?>... attrs) Creates a new directory.
static **Path**	**createFile**(**Path** path, **FileAttribute**<?>... attrs)

java.nio.file.Files

- Le operazioni che prima si svolgevano nella classe File ora sono metodi statici della classe java.nio.file.Files

static boolean	**deleteIfExists**(**Path** path) Deletes a file if it exists.
static boolean	**exists**(**Path** path, **LinkOption**... options) Tests whether a file exists.
static **Stream**<**Path**>	**find**(**Path** start, int maxDepth, **BiPredicate**<**Path**,**BasicFileAttributes**> matcher, **FileVisitOption**... options) Return a Stream that is lazily populated with Path by searching for files in a file tree rooted at
static **Object**	**getAttribute**(**Path** path, **String** attribute, **LinkOption**... options) Reads the value of a file attribute.
static <V extends **FileAttributeView**> V	**getFileAttributeView**(**Path** path, **Class**<V> type, **LinkOption**... options) Returns a file attribute view of a given type.
static **FileStore**	**getFileStore**(**Path** path) Returns the **FileStore** representing the file store where a file is located.
static **FileTime**	**getLastModifiedTime**(**Path** path, **LinkOption**... options) Returns a file's last modified time.
static **UserPrincipal**	**getOwner**(**Path** path, **LinkOption**... options) Returns the owner of a file.
static **Set**<**PosixFilePermission**>	**getPosixFilePermissions**(**Path** path, **LinkOption**... options) Returns a file's POSIX file permissions.
static boolean	**isDirectory**(**Path** path, **LinkOption**... options) Tests whether a file is a directory.

java.nio.file.Files

- Le operazioni che prima si svolgevano nella classe File ora sono metodi statici della classe java.nio.file.Files
- Inclusi metodi di comodo per la creazione di BufferedReader e BufferedWriter

```
try(BufferedReader br = Files.newBufferedReader(Paths.get("myFile.txt"));
        BufferedWriter bw = Files.newBufferedWriter(Paths.get("myFile2.txt")))
{
    // legge da myFile.txt e scrive in myFile2.txt
}
```

🎯Obiettivi

- ✅ Terminale: Print/Output/InputStream, Scanner
- ✅ File: testo, binari
- ✅ Stream
- ✅ Gestione delle risorse
- ✅ Java NIO
  - Serializzazione

Serializzare un oggetto



Quick Response (QR) code



Stream di bit…



Universal Product Code

Serializzare un oggetto in Java

```
import java.io.*;

public class IlMioOggettoSerializzabile
{
    private String nome;
    private int valore;

    public IlMioOggettoSerializzabile(String nome, int valore)
    {
        this.nome = nome;
        this.valore = valore;
    }

    public void salva(String filename)
    {
        try
        {
            FileOutputStream fos = new FileOutputStream(filename);
            ObjectOutputStream os = new ObjectOutputStream(fos);

            os.writeObject(nome);
            os.writeObject(valore);

            os.close();
        }
        catch (IOException e)
        {
            e.printStackTrace();
        }
    }

    public static void main(String[] args)
    {
        new IlMioOggettoSerializzabile("dieci", 10).salva("myObject.ser");
    }
}
```

Costruisce uno stream output di file

Costruisce uno stream output di oggetti

Scrive gli oggetti in formato binario

Chiude lo stream

Domanda: che succede se un campo è un riferimento?

```
public class IlMioOggettoSerializzabile
{
    private String nome;
    private int valore;
    private IlMioOggettoSerializzabile next;

    public IlMioOggettoSerializzabile(String nome, int valore)
    {
        this.nome = nome;
        this.valore = valore;
    }

    public void setNext(IlMioOggettoSerializzabile next)
    {
        this.next = next;
    }

    public void salva(String filename)
    {
        try
        {
            FileOutputStream fos = new FileOutputStream(filename);
            ObjectOutputStream os = new ObjectOutputStream(fos);

            os.writeObject(nome);
            os.writeObject(valore);
            os.writeObject(next);

            os.close();
        }
        catch (IOException e)
        {
            e.printStackTrace();
        }
    }

    public static void main(String[] args)
    {
        IlMioOggettoSerializzabile o1 = new IlMioOggettoSerializzabile("ciao", 2);
        IlMioOggettoSerializzabile o2 = new IlMioOggettoSerializzabile("dieci", 10);
        o2.setNext(o1);
        o2.salva("myObject.ser");
    }
}
```

Riferimento a un (altro) oggetto

Risposta: dipende se l'oggetto in questione è serializzabile oppure no!

- Nell'esempio, otteniamo:

```
java.io.NotSerializableException: IlMioOggettoSerializzabile
        at java.io.ObjectOutputStream.writeObject0(ObjectOutputStream.java:1164)
        at java.io.ObjectOutputStream.writeObject(ObjectOutputStream.java:330)
        at IlMioOggettoSerializzabile.salva(IlMioOggettoSerializzabile.java:29)
        at IlMioOggettoSerializzabile.main(IlMioOggettoSerializzabile.java:44)
```

- Perché la classe non è "serializzabile"
- Per essere serializzabile, deve implementare l'interfaccia Serializable
  - Un'interfaccia **SENZA METODI**! Perché?
  - E' una sorta di marcatore che indica che la classe ha una determinata caratteristica

Tutto o niente!

- Quando si serializza un oggetto, tutti gli oggetti a cui esso si riferisce (variabili d'istanza) vengono serializzati
  - Tutti gli oggetti cui quegli oggetti si riferiscono vengono serializzati
    - Tutti gli oggetti cui quegli oggetti si riferiscono vengono serializzati
      - Tutti gli oggetti cui quegli oggetti si riferiscono vengono serializzati
        - ecc. ecc.!
- Se un solo campo non è serializzabile, l'oggetto non è serializzabile
- Tuttavia è possibile rendere "transiente" un campo con la parola chiave transient
  - Tale campo non sarà serializzato
  - I campi statici sono transient di default

Serial Version UID

- E' bene specificare **sempre** un campo static, final long chiamato serialVersionUID
- Usato in fase di (de)serializzazione per verificare se la versione della classe in uso è la stessa usata per serializzare

```
public class IlMioOggettoSerializzabile implements Serializable
{
    private static final long serialVersionUID = -1327935836496038772L;

    private String nome;
    private int valore;
    private IlMioOggettoSerializzabile next;
```

Identificatore univoco di versione

Leggere un oggetto serializzato: Deserializzazione

```
public static IlMioOggettoSerializzabile leggi(String filename)
{
    try
    {
        // apertura del file
        FileInputStream fis = new FileInputStream(filename);
        ObjectInputStream is = new ObjectInputStream(fis);

        // lettura
        Object o1 = is.readObject();
        Object o2 = is.readObject();
        Object o3 = is.readObject();

        // casting
        String nome = (String)o1;
        int valore = (Integer)o2;
        IlMioOggettoSerializzabile next = (IlMioOggettoSerializzabile)o3;

        // creazione dell'oggetto
        IlMioOggettoSerializzabile o = new IlMioOggettoSerializzabile(nome, valore);
        o.setNext(next);

        is.close();

        return o;
    }
    catch (ClassNotFoundException e)
    {
        e.printStackTrace();
    }
    catch (IOException e)
    {
        e.printStackTrace();
    }

    return null;
}
```

Un tutorial su NIO (non quello di Matrix, quello è Neo)
vs. IO

- http://tutorials.jenkov.com/java-nio/nio-vs-io.html

🎯Obiettivi

- ✅ Terminale: Print/Output/InputStream, Scanner
- ✅ File: testo, binari
- ✅ Stream
- ✅ Gestione delle risorse
- ✅ Java NIO
- ✅ Serializzazione

Credits

Le slide di questo corso sono basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.